



STOCK INSIGHTS

Nifty50 Analytics — Smart Insights for Smarter Investing



Complete Technical Documentation | 2025

Table of Contents

1. Project Overview & Architecture
2. Technology Stack
3. System Architecture Diagram
4. File-by-File Code Documentation
 - 4.1 db_config.py — Database Configuration
 - 4.2 schema.sql — Database Schema
 - 4.3 insert_companies.py — Company Seeder
 - 4.4 data_fetcher.py — Yahoo Finance Sync Engine
 - 4.5 calculations.py — Analytics Engine
 - 4.6 plotting.py — Visualization Helper
 - 4.7 check_data.py — Data Validation Utility
 - 4.8 app.py — Frontend Dashboard (Streamlit)
 - 4.9 sql_bg_queries.sql — Reference SQL
5. Data Flow: End-to-End Journey
6. Project Layer Division (Viva Ready)
 - 6.1 Layer 1 — Backend
 - 6.2 Layer 2 — Backend ↔ Frontend Connectivity
 - 6.3 Layer 3 — Frontend
7. Dashboard Tabs — Feature-by-Feature
8. SQL Scenario Reference
9. Key Formulas & Algorithms
10. Viva Q&A Reference

1. Project Overview & Architecture

Project Purpose

Stock Insights is a real-time Nifty50 stock analytics dashboard that automatically fetches live market data from Yahoo Finance, stores it in a PostgreSQL relational database, and presents advanced technical analysis through an interactive Streamlit web application. The platform is designed to help learners and analysts understand market patterns through visual charts, indicators, risk metrics, and AI-driven insights.

Core Objectives

- Automated daily data ingestion from Yahoo Finance
- Persistent storage in PostgreSQL (relational DB)
- Technical analysis: SMA, EMA, RSI, MACD, Volatility
- Visual dashboards with real-time charts (Plotly)
- Correlation analysis & multi-stock comparison
- AI-trend confidence scoring & price forecasting
- Budget-based affordability calculator

Coverage

- 37 Nifty50 large-cap Indian stocks
- Historical data from January 2015 onwards
- Daily OHLCV (Open, High, Low, Close, Volume)
- Multi-sector coverage: IT, Banking, Energy, FMCG...
- Sector tagging (Energy, Banking, IT, Pharma, etc.)
- Dark/Light theme toggle for UI
- CSV data export from any analysis tab

2. Technology Stack

Function	Purpose	Returns / Notes
<code>Python 3.10+</code>	Primary programming language for all backend logic	Core runtime
<code>PostgreSQL</code>	Relational database for structured OHLCV + company storage	Persistent data store
<code>psycopg2</code>	PostgreSQL adapter for Python — executes all SQL queries	DB driver
<code>yfinance</code>	Yahoo Finance API wrapper — downloads historical stock data	Data source
<code>pandas</code>	Data manipulation, DataFrame operations, rolling calculations	Data processing
<code>NumPy</code>	Numerical computation — trend slope fitting, confidence score	Math engine
<code>Streamlit</code>	Python-based web framework — builds the entire UI dashboard	Frontend framework
<code>Plotly</code>	Interactive charting library — Scatter, Bar, Heatmap charts	Visualization
<code>plotly.express</code>	High-level Plotly interface for correlation heatmaps	Chart helper
<code>logging</code>	Built-in Python logging for data fetch status tracking	Debug/Monitor

3. System Architecture Diagram



High-Level System Flow

Yahoo Finance API → data_fetcher.py → PostgreSQL Database
PostgreSQL Database → calculations.py → analytics results
analytics results → app.py (Streamlit) → User's Browser



Database Schema Structure

TABLE: companies

company_id SERIAL PRIMARY KEY
company_name VARCHAR(100) NOT NULL
ticker_symbol VARCHAR(200) UNIQUE NOT NULL

TABLE: stock_prices

id SERIAL PRIMARY KEY
company_id INTEGER NOT NULL → FK → companies(company_id)
trade_date DATE NOT NULL
open_price NUMERIC(12,4)
high_price NUMERIC(12,4)
low_price NUMERIC(12,4)
close_price NUMERIC(12,4)
volume BIGINT
UNIQUE(company_id, trade_date) ← prevents duplicate daily records
INDEX on (company_id, trade_date) ← fast lookup

4. File-by-File Code Documentation

4.1 db_config.py — Database Configuration



Role & Responsibility

This is the foundational connection module. Every other Python file imports `create_connection()` from here to talk to the PostgreSQL database. It stores all connection credentials and returns a live `psycopg2` connection object.

Configuration Parameters:

Function	Purpose	Returns / Notes
<code>PG_HOST</code>	Database server address	Default: localhost
<code>PG_PORT</code>	PostgreSQL listening port	Default: 5432
<code>PG_DB</code>	Database name to connect to	Value: DemoDb
<code>PG_USER</code>	PostgreSQL username for authentication	Configurable
<code>PG_PASS</code>	PostgreSQL password for authentication	Configurable
<code>create_connection()</code>	Opens and returns a <code>psycopg2</code> connection	Returns conn object

```
# Usage pattern across ALL files:
from db_config import create_connection

conn = create_connection()
with conn.cursor() as cur:
    cur.execute('SELECT * FROM companies;')
    rows = cur.fetchall()
conn.close()
```



Important Notes

- Credentials are stored as plain text — use environment variables in production
- Every function that calls `create_connection()` must explicitly close the connection
- `psycopg2` connections are NOT thread-safe — create a new one per request/operation

4.2 schema.sql — Database Schema



Role & Responsibility

This SQL file defines the complete database structure. It is run ONCE to initialize the PostgreSQL database with two tables: `companies` (master) and `stock_prices` (fact).

Function	Purpose	Returns / Notes
<code>companies</code>	Master table — stores company metadata	<code>company_id</code> (PK), name, ticker
<code>stock_prices</code>	Fact table — stores daily OHLCV price data	Linked to companies via FK
<code>UNIQUE constraint</code>	Prevents duplicate entries per company per day	(<code>company_id</code> , <code>trade_date</code>)
<code>CASCADE DELETE</code>	Deleting a company auto-deletes all its price rows	Referential integrity
<code>INDEX</code>	Speeds up filtered queries by company + date	<code>idx_stockprices_company_date</code>

🔑 Design Decisions

- `DROP TABLE IF EXISTS` used at top — safe re-initialization during development
- `stock_prices.id` uses `SERIAL` (auto-increment) as primary key
- `NUMERIC(12,4)` for prices allows 8 digits before decimal, 4 after
- The `UNIQUE(company_id, trade_date)` constraint allows safe UPSERT operations

4.3 insert_companies.py — Company Seeder

📋 Role & Responsibility

Seeds the companies table with 37 Nifty50 stocks. Uses PostgreSQL's `ON CONFLICT` clause for idempotency — safe to run multiple times without creating duplicates. Can be run standalone or is auto-triggered by `data_fetcher.py` if DB is empty.

Function	Purpose	Returns / Notes
<code>DEFAULT_COMPANIES</code>	List of 37 (name, ticker) tuples	Static configuration
<code>insert_companies()</code>	Upserts all companies into DB	Returns inserted count
<code>ON CONFLICT</code>	Updates company name if ticker already exists	Idempotent operation
<code>xmax = 0 check</code>	Distinguishes new inserts from updates in PG	PostgreSQL-specific trick

```
# Idempotent upsert pattern:
INSERT INTO companies (company_name, ticker_symbol)
VALUES (%s, %s)
ON CONFLICT (ticker_symbol)
DO UPDATE SET company_name = EXCLUDED.company_name
RETURNING (xmax = 0) AS is_new; -- True if new row was inserted
```

4.4 data_fetcher.py — Yahoo Finance Sync Engine

Role & Responsibility

This is the data ingestion backbone. It fetches daily OHLCV stock data from Yahoo Finance using the yfinance library and upserts it into PostgreSQL. It tracks the latest date already stored per ticker, avoids redundant downloads, and runs automatically via a 24-hour Streamlit cache trigger in app.py.

Function	Purpose	Returns / Notes
<code>safe_float(v)</code>	Safely converts any value to float	Returns None on error
<code>safe_int(v)</code>	Safely converts to int via float intermediary	Returns 0 on error
<code>get_company_list()</code>	Fetches tickers from DB; seeds if empty	Returns list of strings
<code>get_latest_date(t)</code>	Finds newest trade_date stored for ticker	Returns date or None
<code>fetch_yfinance(t,d)</code>	Downloads OHLCV from Yahoo Finance API	Returns pandas DataFrame
<code>insert_prices(df,t)</code>	Upserts rows into stock_prices table	Handles MultiIndex columns
<code>run_fetching()</code>	Orchestrates full update for all tickers	Main entry point

Smart Incremental Sync Logic

1. Check latest trade_date already in DB for each ticker
2. If data exists → fetch from (latest_date - 7 days) for safety overlap
3. If no data exists → fetch from START_DATE (Jan 2015)
4. Upsert with ON CONFLICT → no duplicates, always up-to-date
5. 0.3 second pause between tickers to respect Yahoo Finance rate limits
6. Streamlit `@st.cache_resource(ttl=86400)` ensures once-per-day execution

```
# yfinance download call:
df = yf.download(
    ticker,
    start=start_date.strftime('%Y-%m-%d'),
    end=today.strftime('%Y-%m-%d'),
    interval='1d',           # daily bars
    auto_adjust=True,        # adjusts for splits/dividends
    progress=False,
)

# PostgreSQL UPSERT:
INSERT INTO stock_prices (company_id, trade_date, open_price, ...)
VALUES (%s, %s, %s, ...)
ON CONFLICT (company_id, trade_date) DO UPDATE SET
    close_price = EXCLUDED.close_price, ...;
```

4.5 calculations.py — Analytics Engine

Role & Responsibility

The most complex and central file. Contains all data-fetching queries, technical indicator calculations, risk metrics, and multi-stock comparison logic.

All functions receive/return pandas DataFrames and are used directly by app.py.

Data Retrieval Functions

Function	Purpose	Returns / Notes
<code>fetch_prices(t,s,e)</code>	Fetches full OHLCV history for a ticker with optional date range	Returns DataFrame or None
<code>fetch_current_price(t)</code>	Gets only the latest closing price for a ticker	Returns float or None
<code>fetch_company_info(t)</code>	Returns company name and ticker symbol dict	Returns dict or None
<code>get_close_price_column(df)</code>	Finds the close price column (case-insensitive)	Returns column name string

Technical Indicator Functions

Function	Purpose	Returns / Notes
<code>compute_sma(df, window)</code>	Simple Moving Average — rolling mean over N days	Adds 'SMA' column
<code>compute_ema(df, window)</code>	Exponential Moving Average — weighted recent prices more	Adds 'EMA' column
<code>add_technical_indicators(df)</code>	Full indicator suite — SMA 20/50/200, EMA 20/50, RSI 14, MACD 12/26/9, Golden Cross	Adds 9 indicator columns
<code>detect_abrupt_changes(df, thresh)</code>	Finds days with % price change above threshold	Returns filtered DataFrame
<code>volatility_and_risk(df, window)</code>	Rolling standard deviation (volatility) + risk ratio	Adds 2 columns

Comparison & Strategy Functions

Function	Purpose	Returns / Notes
<code>correlation_analysis(tickers)</code>	Computes pairwise Pearson correlation of close prices	Returns correlation matrix
<code>compare_companies(tickers,s,e)</code>	Merges close prices for multiple tickers by date	Returns merged DataFrame
<code>best_time_to_invest(df)</code>	Returns rows where price > SMA (bullish momentum)	Returns filtered DataFrame

RSI Calculation Formula

RSI (Relative Strength Index) — 14-day period

1. $\text{delta} = \text{close.diff()}$ (daily change)

```

2. gain    = delta where delta > 0, else 0
3. loss    = -delta where delta < 0, else 0
4. avg_gain = rolling(14).mean() of gain
5. avg_loss = rolling(14).mean() of loss
6. RS      = avg_gain / avg_loss
7. RSI     = 100 - (100 / (1 + RS))
Interpretation: RSI > 70 = Overbought | RSI < 30 = Oversold

```

MACD Calculation Formula

MACD (Moving Average Convergence Divergence)

```

1. EMA_12 = close.ewm(span=12, adjust=False).mean()
2. EMA_26 = close.ewm(span=26, adjust=False).mean()
3. MACD Line = EMA_12 - EMA_26
4. Signal   = MACD_line.ewm(span=9, adjust=False).mean()
5. Histogram = MACD - Signal (momentum divergence)
Bullish: MACD crosses above Signal | Bearish: MACD crosses below Signal

```

4.6 plotting.py — Visualization Helper

Role & Responsibility

A lean utility module containing the correlation heatmap rendering function.
Uses Plotly Express imshow to render a color-coded matrix of stock correlations.

Function	Purpose	Returns / Notes
<code>plot_correlation(corr_matrix)</code>	Renders an interactive Plotly heatmap using Viridis scale	Displays inline via <code>st.plotly_chart</code>
<pre> # Heatmap uses Viridis color scale: fig = px.imshow(corr_matrix, text_auto=True, # shows numeric values in cells color_continuous_scale ='Viridis',) st.plotly_chart(fig, use_container_width=True) </pre>		

4.7 check_data.py — Data Validation Utility

Role & Responsibility

A developer/diagnostic script used to verify database health.
Run manually via terminal to confirm companies are seeded and price data is present.
Not called from the main app — it is a standalone verification tool.

Function	Purpose	Returns / Notes
<code>check_companies()</code>	Prints all companies stored in DB as a list of dicts	Console output
<code>check_price_counts()</code>	Prints count of price rows per ticker	Console output

4.8 app.py — Streamlit Frontend Dashboard

Role & Responsibility

The entry point and user-facing layer of the application. It bootstraps the Streamlit interface, handles all user interaction, wires up backend calculations, and renders 5 specialized analytical tabs. This file bridges all other modules.

Initialization & Global Setup

Function	Purpose	Returns / Notes
<code>st.set_page_config()</code>	Sets browser tab title, icon, layout=wide, sidebar expanded	Must be first Streamlit call
<code>sector_map dict</code>	Maps each ticker to its business sector with emoji labels	Used in sidebar display
<code>theme toggle</code>	Dark/Light mode via sidebar radio button	CSS variable injection
<code>silent_update()</code>	Calls <code>run_fetching()</code> once per 24hrs via <code>@cache_resource</code>	Auto data sync

Sidebar Controls

Function	Purpose	Returns / Notes
<code>multiselect tickers</code>	User selects 1–N companies to analyze	drives all 5 tabs
<code>date_input start/end</code>	Date range filter applied to all queries	MIN_DATE = 2015-01-01
<code>staleness check</code>	Computes days since last update per ticker	Shows warning if >2 days old
<code>sector labels</code>	Displays sector classification per selected stock	Informational sidebar card

Helper Functions in app.py

Function	Purpose	Returns / Notes
----------	---------	-----------------

<code>build_budget_options()</code>	Generates ₹0–₹10L budget increments for selectbox	Returns (values, labels) lists
<code>analyze_trend_confidence(df, col, horizon)</code>	Scores trend as Buy/Hold/Risky based on slope + pct change + volatility	Returns (conf%, label, pct, vol)
<code>project_future(df, col, horizon)</code>	Linear regression on recent close prices to project next 15/60 days	Returns (buy_df, sell_df)
<code>download_csv(df, name)</code>	Renders st.download_button for any DataFrame	CSV export helper

5 Dashboard Tabs

Function	Purpose	Returns / Notes
Tab 1 – Price Trends	Overlay/panel view of SMA/EMA + RSI/MACD oscillators	Plotly interactive charts
Tab 2 – Abrupt Changes	Bar chart of daily % changes beyond a slider threshold	Color-coded green/red bars
Tab 3 – Risk & Volatility	Rolling std deviation + risk ratio line chart	Window slider control
Tab 4 – Compare & Correlate	Normalized price comparison + Pearson correlation heatmap	Requires ≥2 companies
Tab 5 – Smart Insights	AI trend score, buy/sell zone forecast, budget calculator	Linear regression forecast

4.9 sql_bg_queries.sql — Reference SQL



Role & Responsibility

A developer reference file containing example SQL queries for the most common data retrieval scenarios. These are the raw SQL equivalents of what Python's `calculations.py` generates dynamically. Used for DB inspection and learning.

Function	Purpose	Returns / Notes
Scenario 1	Fetch OHLCV for RELIANCE.NS from 2024-01-01	Filtered by company + start date
Scenario 2	Get the single most recent close price for a ticker	ORDER BY date DESC LIMIT 1
Scenario 3	Fetch full year OHLCV between two dates (BETWEEN)	Used for date range comparisons
Scenario 4	MAX(trade_date) per company	Used by <code>get_latest_date()</code>

5. Data Flow: End-to-End Journey

Step 1	Streamlit launches app.py. @cache_resource triggers silent_update() once per 24 hours.
Step 2	get_company_list() reads ticker symbols from PostgreSQL companies table.
Step 3	get_latest_date(ticker) queries MAX(trade_date) per ticker to find gaps.
Step 4	fetch_yfinance() downloads new OHLCV rows from Yahoo Finance API.
Step 5	insert_prices() runs ON CONFLICT upsert for each downloaded row.
Step 6	User picks tickers and date range from Streamlit sidebar.
Step 7	fetch_prices() builds parameterized SQL and returns a pandas DataFrame.
Step 8	add_technical_indicators() computes SMA/EMA/RSI/MACD on the DataFrame.
Step 9	Plotly charts are rendered inside Streamlit tabs interactively in browser.
Step 10	User can download any analysis result as CSV via download_csv().

6. Project Layer Division (Viva Ready)

The project is cleanly divided into 3 distinct architectural layers. Understanding this division is essential for explaining the project in a viva/exam scenario.



LAYER 1 — BACKEND

Files: `db_config.py` | `schema.sql` | `insert_companies.py` | `data_fetcher.py`

The Backend is the invisible engine room of the application. The user NEVER directly interacts with any of these files. They collectively handle:

- Database configuration and connection management
- Table creation and schema initialization
- Company data seeding
- Automated daily data ingestion from Yahoo Finance
- Data persistence via PostgreSQL upsert operations

Function	Purpose	Returns / Notes
<code>db_config.py</code>	Manages all DB credentials and connection factory	Foundation of all DB operations
<code>schema.sql</code>	Defines table structures, constraints, and indexes	Run once to initialize DB
<code>insert_companies.py</code>	Seeds the companies master table with Nifty50 tickers	Idempotent — safe to re-run
<code>data_fetcher.py</code>	Fetches Yahoo Finance data and writes to <code>stock_prices</code>	Auto-runs via Streamlit cache



Viva Explanation (Layer 1)

'The backend is responsible for data acquisition and storage. `db_config.py` provides a centralised connection factory using `psycopg2`. `schema.sql` initialises two tables — `companies` as the master and `stock_prices` as the fact table with a foreign key relationship. `insert_companies.py` seeds the master table with 37 Nifty50 tickers using an idempotent `ON CONFLICT UPSERT`. `data_fetcher.py` is the core sync engine — it checks the latest stored date per ticker, fetches only missing data from Yahoo Finance using `yfinance`, and writes to PostgreSQL using `ON CONFLICT UPSERT` to avoid duplicates. This entire backend is completely decoupled from the UI.'



LAYER 2 — BACKEND ↔ FRONTEND CONNECTIVITY

Files: `calculations.py` | `plotting.py` | `check_data.py`

The Connectivity Layer is the bridge between raw stored data and the visual UI.

It reads data from PostgreSQL, applies all mathematical transformations, and returns structured pandas DataFrames ready for Streamlit to render.

- calculations.py is the brain: all SQL queries + all indicator math live here
- plotting.py provides reusable chart components for the heatmap
- check_data.py provides DB health diagnostics (developer utility)

Function	Purpose	Returns / Notes
<code>calculations.py</code>	SQL queries + SMA/EMA/RSI/MACD/Volatility/Correlation logic	Returns DataFrames to app.py
<code>plotting.py</code>	<code>plot_correlation()</code> renders interactive Plotly heatmap	Called by app.py Tab 4
<code>check_data.py</code>	Developer tool to inspect DB contents from terminal	Not called by app.py

Viva Explanation (Layer 2)

'The connectivity layer is the data pipeline between the database and the UI. calculations.py contains parameterized SQL queries that fetch OHLCV data into pandas DataFrames. These DataFrames are then enriched with technical indicators computed using NumPy and pandas — SMA via `rolling().mean()`, EMA via `ewm().mean()`, RSI using a 14-day gain/loss ratio, and MACD via the 12-26-9 EMA difference. The resulting enriched DataFrames are returned directly to app.py which renders them as Plotly charts. This separation means the UI layer has ZERO SQL knowledge — it only works with pandas DataFrames. plotting.py further separates the heatmap rendering concern into its own reusable component.'

LAYER 3 — FRONTEND

Files: app.py (Streamlit Dashboard)

The Frontend is entirely contained within app.py — a Python file rendered as a reactive web application by Streamlit. It handles:

- Page configuration, SEO meta tags, dark/light theme injection
- Sidebar: company selector, date pickers, staleness alerts
- 5 analytical tabs with interactive Plotly charts
- Helper logic: trend scoring, linear regression forecast, budget calculator
- CSV export for any data view
- Auto-refresh data sync via `@st.cache_resource`

Function	Purpose	Returns / Notes
<code>Page Config</code>	Sets browser tab, icon, layout=wide, dark theme	First call in app.py
<code>Theme Toggle</code>	Dark/Light CSS variables injected via <code>st.markdown</code> HTML	Sidebar radio widget

<code>auto-sync</code>	<code>@cache_resource(ttl=86400)</code> → runs data fetch once/day	Transparent to user
Sidebar Filters	multiselect + date_input → drives all 5 tab queries	Global filter state
Tab 1: Price Trends	Plotly Scatter with selectable MA overlays + RSI/MACD subplots	<code>go.Figure()</code>
Tab 2: Abrupt Changes	Slider threshold → <code>px.bar</code> with <code>RdYlGn</code> color scale	<code>px.bar()</code>
Tab 3: Risk	<code>st.line_chart</code> with volatility + risk dual series	Streamlit native chart
Tab 4: Compare	<code>go.Figure</code> multi-trace + correlation matrix heatmap	Requires ≥ 2 tickers
Tab 5: Smart Insights	Trend confidence score + linear regression buy/sell zones	NumPy polyfit

Viva Explanation (Layer 3)

'The frontend is a Streamlit-based web application defined entirely in `app.py`. Streamlit converts Python code into an interactive browser UI without any HTML, CSS or JavaScript knowledge needed. The sidebar hosts the global filters — company multiselect and date range pickers — which are passed to `calculations.py` functions as parameters. The 5 tabs each represent a distinct analysis module. Tab 1 renders candlestick-style price charts with toggleable SMA/EMA overlays. Tab 2 visualizes abrupt price movements. Tab 3 shows rolling volatility and risk. Tab 4 compares multiple stocks normalized and visualizes their Pearson correlation. Tab 5 applies NumPy linear regression to project future price tendency and labels buy/sell zones. Custom CSS is injected via `st.markdown(unsafe_allow_html=True)` for styling cards.'

7. Dashboard Tabs — Feature by Feature

Tab 1 — Price Trends

Displays historical closing prices with selectable Moving Average overlays.

Indicators: SMA 20/50/200, EMA 20/50

Oscillators: RSI (14), MACD

Modes: Overlay on Chart OR Separate Panels

Shows: Latest Close Price metric, Download CSV

Tab 2 — Abrupt Changes

Detects daily percentage moves exceeding a user-defined threshold (3–20%).

Bar chart colored on RdYlGn scale (green=positive, red=negative).

Useful for spotting news-driven events or institutional buying.

Tab 3 — Risk & Volatility

Rolling standard deviation of close prices over a sliding window (5–55 days).

Risk ratio = volatility / close_price

Line chart: dual-series volatility + risk over full history

Tab 4 — Compare & Correlate

Merges close prices across selected tickers and plots normalized comparison.

Pearson correlation matrix rendered as Viridis heatmap via plotting.py.

Values near 1.0 = move together | near -1.0 = move opposite.

Tab 5 — Smart Insights (Most Advanced Tab)

- Trend Confidence Score: Based on slope of recent price (NumPy polyfit) + % change + volatility
 - Signal Labels: Strong Buy / Buy / Hold / Risky Avoid — based on slope and pct change
 - Forecast: Linear regression extended 15 days (short-term) or 60 days (long-term)
 - Buy Zones: Future projected prices where close < SMA → undervalued entry signal
 - Sell Zones: Future projected prices where close > SMA → overvalued exit signal
 - Budget Calculator: Divides user's ₹ budget by current price to estimate share count
-  All output is for educational purposes only — not financial advice

8. SQL Scenario Reference

These are the actual SQL patterns used by `calculations.py` mapped to real dashboard scenarios:

```
-- SCENARIO 1: Fetch OHLCV for a specific company from a start date
-- Triggered when: user selects RELIANCE.NS + sets start date = 2024-01-01
SELECT sp.trade_date, sp.open_price, sp.high_price, sp.low_price,
       sp.close_price, sp.volume
FROM stock_prices sp
JOIN companies c ON sp.company_id = c.company_id
WHERE c.ticker_symbol = 'RELIANCE.NS'
      AND sp.trade_date >= '2024-01-01'
ORDER BY sp.trade_date ASC;
```

```
-- SCENARIO 2: Fetch the latest/most recent close price
-- Triggered by: fetch_current_price() → used in Smart Insights tab
SELECT sp.close_price
FROM stock_prices sp
JOIN companies c ON sp.company_id = c.company_id
WHERE c.ticker_symbol = 'RELIANCE.NS'
ORDER BY sp.trade_date DESC
LIMIT 1;
```

```
-- SCENARIO 3: Date-range comparison for multiple companies
-- Each ticker is fetched individually, then merged by pandas in Python
SELECT sp.trade_date, sp.close_price
FROM stock_prices sp
JOIN companies c ON sp.company_id = c.company_id
WHERE c.ticker_symbol = 'RELIANCE.NS'
      AND sp.trade_date BETWEEN '2023-01-01' AND '2023-12-31'
ORDER BY sp.trade_date ASC;
```

```
-- SCENARIO 4: Find the latest available data date (used by incremental sync)
SELECT MAX(trade_date) FROM stock_prices
WHERE company_id = (SELECT company_id FROM companies WHERE ticker_symbol =
' TCS.NS');
```

9. Key Formulas & Algorithms

Function	Purpose	Returns / Notes
<code>SMA (N)</code>	Sum of last N closing prices / N	Rolling window mean
<code>EMA (N)</code>	<code>close.ewm(span=N, adjust=False).mean()</code>	Exponentially weighted, recent bias
<code>RSI (14)</code>	$100 - (100 / (1 + \text{avg_gain} / \text{avg_loss}))$	Momentum oscillator 0–100
<code>MACD</code>	$\text{EMA}(12) - \text{EMA}(26)$; Signal = $\text{EMA}(9)$ of MACD	Trend momentum divergence
<code>Volatility</code>	<code>close.rolling(N).std()</code>	N-day standard deviation
<code>Risk Ratio</code>	<code>volatility / close_price</code>	Normalised risk per price unit
<code>Golden Cross</code>	1 if <code>SMA_50 > SMA_200</code> else 0	Bull market signal
<code>pct_change</code>	$(\text{close}[t] - \text{close}[t-1]) / \text{close}[t-1]$	Daily percentage movement
<code>Trend Conf.</code>	$50 + (\text{pct_change}/2) - (\text{volatility}/4)$, clipped to [5, 95]	Composite confidence score
<code>Forecast</code>	<code>np.polyfit(x, y, 1) → slope + intercept</code> → extend N business days	Linear regression projection

10. Viva Q&A Reference

Q1 What database does this project use, and why?

↳ PostgreSQL — a production-grade relational RDBMS. It offers ACID compliance, strong SQL support, SERIAL auto-increment PKs, ON CONFLICT UPSERT (essential for idempotent data sync), and efficient indexing for time-series queries.

Q2 Why was yfinance chosen as the data source?

↳ yfinance provides free, easy-to-use access to Yahoo Finance's historical daily OHLCV stock data via a clean Python API. It supports `auto_adjust=True` for split/dividend-adjusted prices and handles multi-ticker downloads efficiently.

Q3 How does the app prevent duplicate data in the database?

↳ By using PostgreSQL's ON CONFLICT (company_id, trade_date) DO UPDATE SET clause — this is an UPSERT. If a row for that company and date already exists, it updates the price fields instead of failing or creating a duplicate.

Q4 What is the role of psycopg2?

↳ psycopg2 is the Python database adapter for PostgreSQL. It translates Python function calls into PostgreSQL wire protocol, handles parameterized queries (preventing SQL injection), manages transactions, and returns results as Python tuples.

Q5 How does Streamlit work?

↳ Streamlit is a Python framework that converts Python scripts into interactive web apps. It re-runs the entire script top-to-bottom on every user interaction, maintaining state via `st.session_state`. No HTML/CSS/JS knowledge is required.

Q6 Explain the data flow from Yahoo Finance to the chart.

↳ Yahoo Finance → `yfinance.download()` → pandas DataFrame → psycopg2 INSERT → PostgreSQL → `fetch_prices()` SQL query → pandas DataFrame → `add_technical_indicators()` → `Plotly go.Figure()` → `st.plotly_chart()` → Browser

Q7 What is RSI and how is it computed here?

↳ RSI (Relative Strength Index) measures momentum. Computed as: $100 - 100 / (1 + \text{avg_gain} / \text{avg_loss})$ over a 14-day window. $\text{RSI} > 70$ signals overbought (possible correction), $\text{RSI} < 30$ signals oversold (possible bounce).

Q8 What is the difference between SMA and EMA?

↳ SMA (Simple Moving Average) gives equal weight to all N days — slow to react. EMA (Exponential Moving Average) weights recent days more heavily via exponential decay — reacts faster to recent price changes.

Q9 How does the Smart Insights tab produce buy/sell zones?

↳ It applies NumPy's polyfit (linear regression) on the most recent N days of close prices to compute slope + intercept. This line is extended for 15/60 future business days. Days where projected price < SMA are buy zones; > SMA are sell zones.

Q10 Why is the @st.cache_resource decorator used?

↳ cache_resource(ttl=86400) ensures that the expensive run_fetching() data sync function runs at most once every 24 hours (86400 seconds) regardless of how many users interact with the app, preventing redundant Yahoo Finance API calls.

Q11 What is the Golden Cross indicator?

↳ Golden Cross = 1 when SMA_50 crosses above SMA_200. It signals a long-term bullish trend — short-term momentum overtaking long-term average. Used by institutional investors as a buy confirmation signal.

Q12 How does the incremental data sync work?

↳ get_latest_date() queries MAX(trade_date) per ticker. New fetch starts from (latest_date - 7 days) as a safety overlap to recover any missed days. If no data exists, fetch starts from January 2015. This avoids full re-downloads.

**Stock Insights — Complete Documentation**

Python • PostgreSQL • Streamlit • Plotly • Yahoo Finance • Pandas • NumPy